String Comparison In Java

27 December 2025 10:16

String comparison is a key component of Java programming and is used extensively in reference matching, sorting and authentication.

We can compare String in Java on the basis of content and reference.

String comparison used in authentication (by the equals() method), sorting(compareTo() method), reference matching (by the == operator)

There are three ways to compare string in Java:

1. By using equals() method
2. By using compareTo() method
3. Using startsWith() and endsWith() Method

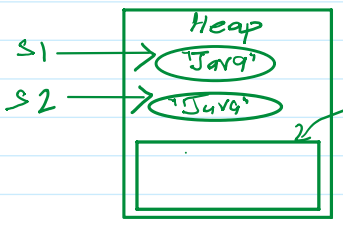
1. **By using equals() method:** The String class equals () method compares the original content of the string. It compares values of string for equality. String class provides the following two methods: Additionally, the equalsIgnoreCase() method performs a case insensitive comparison.

   String class provides the following two methods:

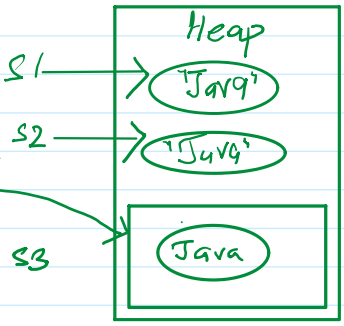
- **public boolean equals(Object another)** Compares this string to the specified object.
- **public boolean equalsIgnoreCase(String another)** Compares this string to another string, ignoring case.

Examples:

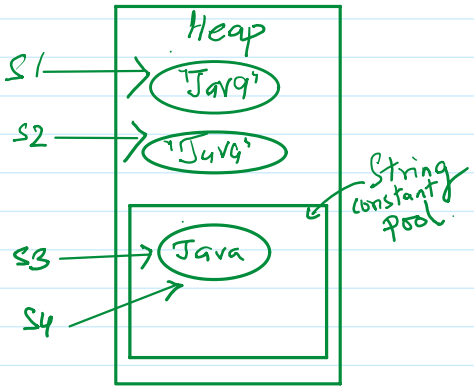
```
String s1 = new String("Java");
String s2 = new String("Java");
```




```
String s1 = new String("Java");
String s2 = new String("Java");
String s3 = "Java";
```

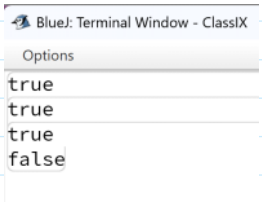



```
String s1 = new String("Java");
String s2 = new String("Java");
String s3 = "Java";
String s4 = "Java";
```




```
String s1 = new String("Java");
String s2 = new String("Java");
String s3 = "Java";
String s4 = "Java";
String s5 = "C++";
System.out.print("\f"+s1.equals(s2));
System.out.print("\n"+s3.equals(s2));
System.out.print("\n"+s3.equals(s4));
System.out.print("\n"+s3.equals(s5));
```

Output:



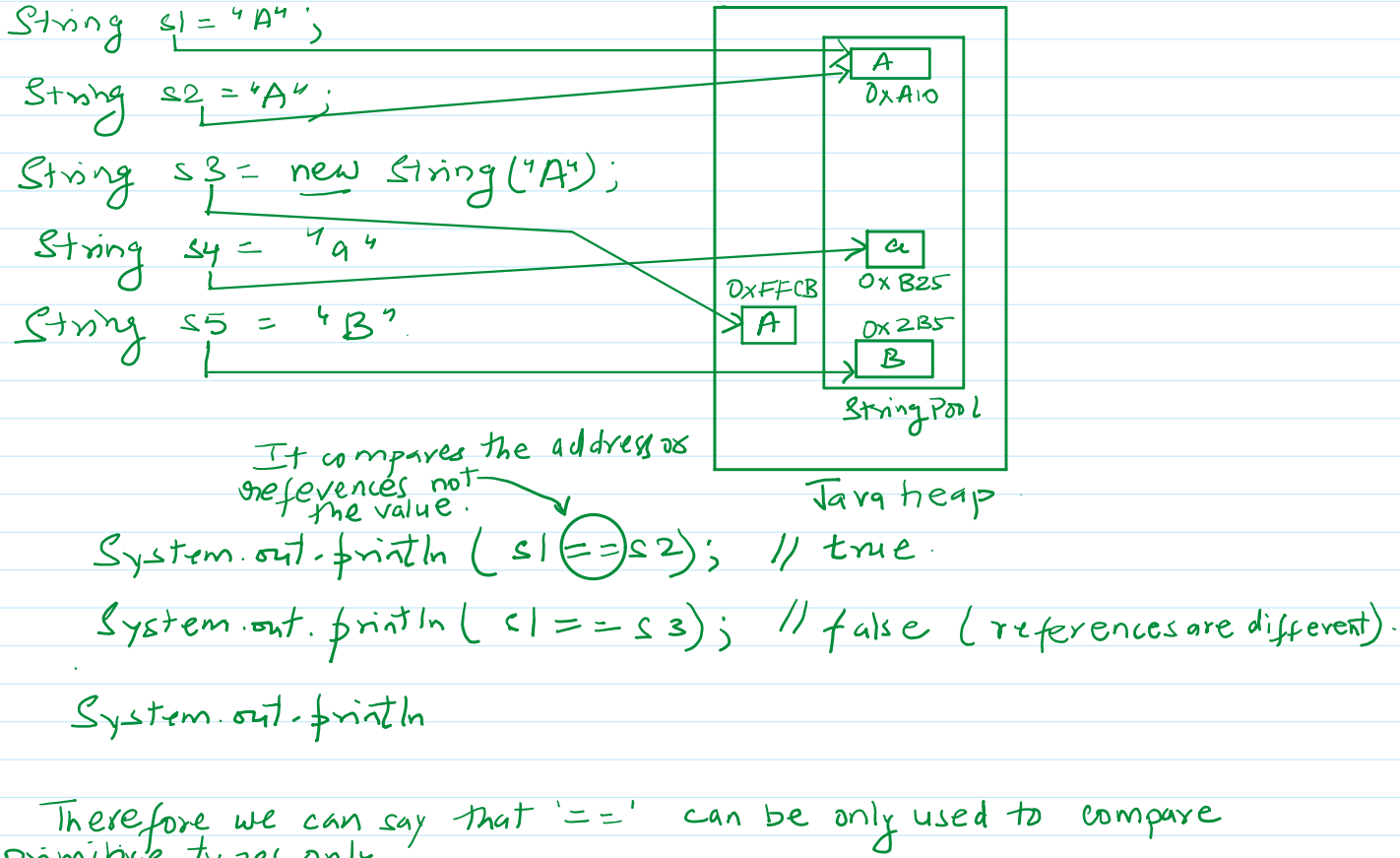
Example of equalsIgnoreCase(String obj):

```
String name1 = "Ram";
String name2 = "rAm";

boolean res = name1.equalsIgnoreCase(name2);
res
true   (boolean)
```

By using == Operator

In Java, the == operator compares references rather than values.



Therefore we can say that '==' can be only used to compare primitive types only.

String compare by compareTo() method

compare is a search technique to find a word in dictionary.

The String class **compareTo()** method compares values lexicographically and returns an integer value that describes if first string is less than, equal to or greater than 2nd string.

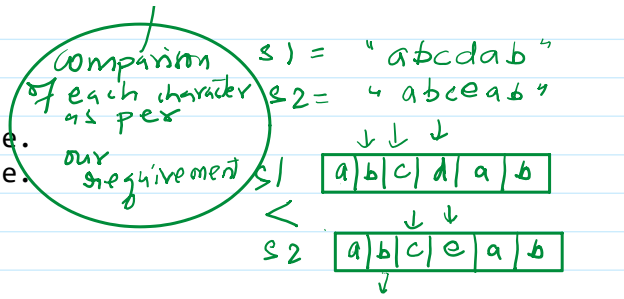
Comparison of each character s1 = "abcdab" s2 = "abceab"

Suppose S1 and S2 are two String objects. If:

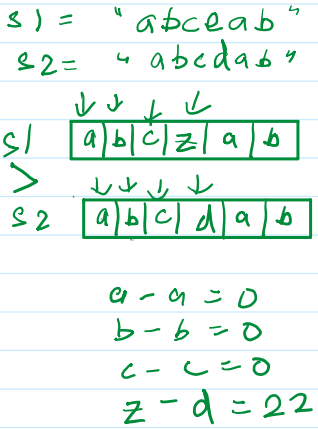
- S1 == S2: the method returns 0

Suppose S1 and S2 are two String objects. If:

- S1 == S2: the method returns 0.
- S1 > S2: The method returns a positive value.
- S1 < S2: The method returns a negative value.




```
if (s1.compareTo(s2) == 0) {
    ==
    =
}
```




```
if (s1.compareTo(s2) > 0) {
    ==
    =
}
if ( s1.compareTo(s2) < 0) {
    ==
    =
}
```

```
String s1 = "abczef";
String s2 = "abcde";
System.out.println(s1.compareTo(s2));
System.out.println(s2.compareTo(s1));
String s3 = "abcdfe";
s1
"abczef"   (String)
System.out.println(s1.compareTo(s3));
System.out.println(s1.compareTo(s1));
```

Using startsWith() and endsWith() method

If a string starts with a prefix and ends with a suffix, respectively, it may be determined using the methods startsWith()and endsWith().

```
String s1 = "aaabbbccc";
String s2 = "aaa";
String s3 = "ccc";
String s4 = "bbb";
System.out.println(s1.startsWith(s2));
System.out.println(s1.endsWith(s3));
System.out.println(s1.endsWith(s4));
```

```
System.out.println(s1.startsWith(s4));
```

Output:

```
true
true
false
false
```

Finding number of characters in a String by using length() method:

```
s1
"aaabbbccc"   (String)
System.out.println("Number of characters: " + s1.length());
int l = s1.length();
l
9   (int)
```

Output:

```
Number of characters: 9
```